Template updates for audit engagements

Publish once per product and market. Resolve the badge from verified metadata when the auditor lists files. Reopening 20,000 files at 20 concurrent one-minute loads would take 16 h 40 min.

One release, from publication to the auditor's screen / Figure 1

Step	What happens to a Canadian-market release
1. Publish v7	Publish v7, allowed upgrade paths and outbox revision 42 in one shared template DB transaction.
2. Distribute	Regional consumers ignore older revisions and fetch a snapshot on gaps. Prepare common summaries in the background.
3. List 50 files	Read the local catalog and firm index. An auditor who declined v5 for a file on v4 sees the v7 offer. A file on v7 is current.
4. Inspect changes	Resolve the exact hashes, locale and policy. Pin the archived text to a view receipt. On a miss, show "Preparing summary" and enqueue one job for that pair.

Data ownership and tenant isolation

Template service. The shared DB stores all template versions as ZIPs of structured JSON. Lookup by ID and version is fast. Content owners govern immutable releases, parent links, upgrade paths and withdrawals by market. It holds only shared template data.

Engagement manager. On create, it reads the latest applicable template and saves its ID and version inside the file. Its new hook also writes a queryable row in the firm DB (0.8 GB raw at 1 KB/file). The same manager opens files and records decisions. Only confirmed application advances the effective version.

Regional API. Derive the firm and database from the authenticated session. Authorize files on lists and detail views. Exclude exact declines, pin the catalog revision and page by product, track, baseline and file ID. Firm data, receipts, queues, logs, keys and backups stay in-region. Only template data and opaque admission tokens cross regions.

Advantages and disadvantages of the main decisions

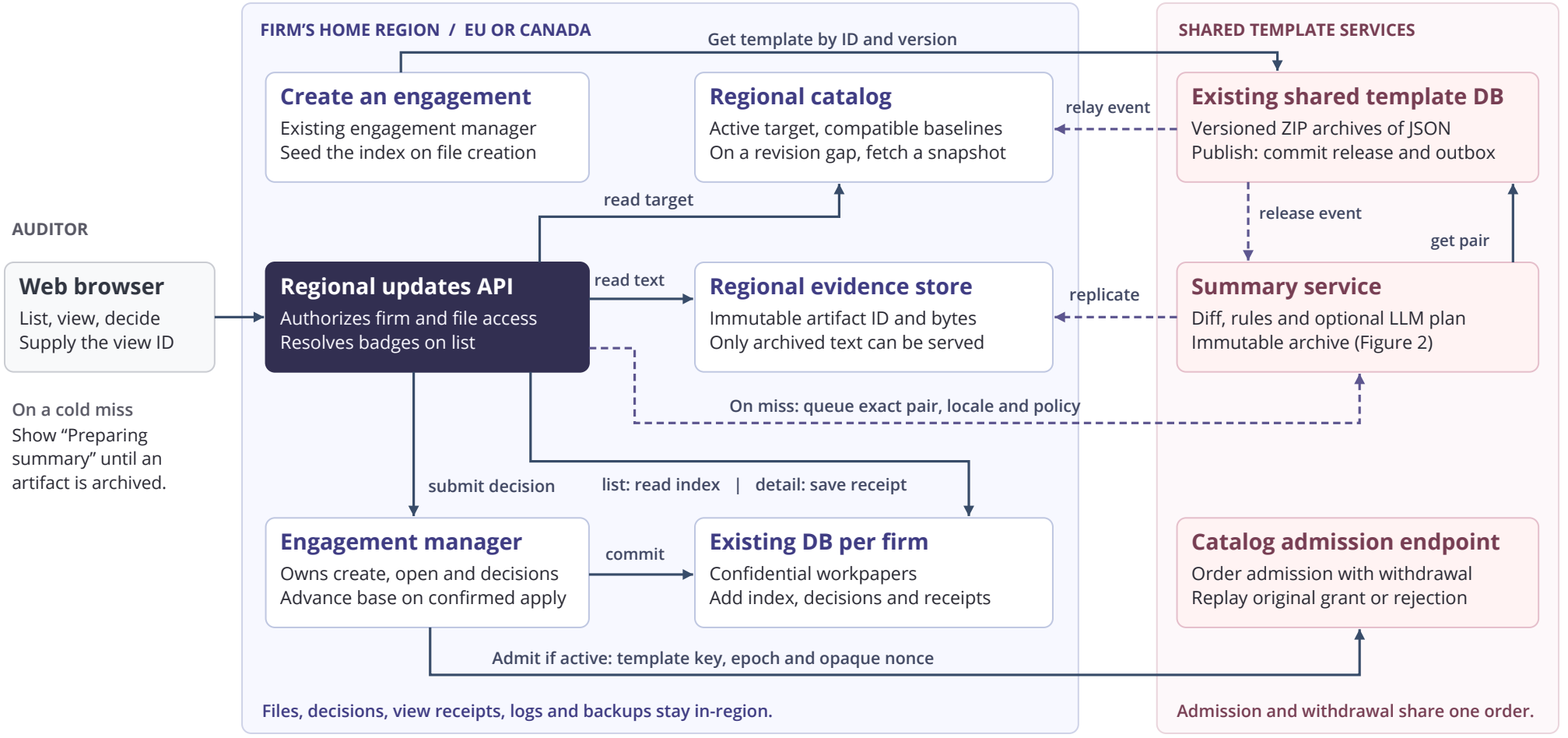
Decision	Advantage	Disadvantage
Distribute catalog events	The outbox retains each revision until the relay can deliver it.	While delivery is pending, a regional list may show the old target.
Compute badges on read	One catalog change makes v7 discoverable without writing to every file.	Sparse pending lists may scan many rows. Measure rows scanned per result.
Check decisions with the template authority	Apply and decline requests have a clear order relative to a withdrawal.	A decision adds a service call. An authority outage pauses new decisions.
Limit background verification	The file service retains capacity for live engagements during migration.	Some legacy files wait for verification. At 20 slots, a full backfill takes about 28 days.

The one requirement I would challenge

Day-one freshness for every file. "Created from v4" leaves the current version uncertain. Guessing risks showing the wrong changes. Opening the file takes about a minute. I would agree with Product to show updates within seconds for verified files. Until the manager confirms a legacy file's version, show "Needs verification" and withhold its version-specific offer.

Scope. Content application, automatic branch merges and audit conclusions are outside this feature.

End-to-end system architecture



BACKGROUND VERIFICATION / SAME FIRM REGION

Exception planner + SQS
On inventory or catalog revision
Only unknown or flagged baselines

lease job

Java 25 fan-out worker
Check completion on job receipt
Reuse the same operation ID

start / status

Session pods / same manager
Rehydrate workpapers in memory (~60 s)
Pod derives volatile financial values

Owner Write verified metadata only if the file revision matches.

Worker Acknowledge SQS after durable completion.

Recovery On timeout, query the operation ID. The manager holds a shared slot until completion or enforced cancellation.

→ service or database call

⇢ event, queue or replication

Which update should the auditor see?

Select the content team's active target for the product and market. Require an approved path from the effective baseline. Hold offers with ambiguous targets or missing compatibility data.

Event for one engagement	What the auditor sees or can do
v4 applied, v5 declined	The decline pins this engagement, baseline v4 and immutable target v5. Another file at the same firm can still accept v5.
v6 and v7 accumulate	If v7 is the approved compatible target, show one cumulative summary from v4 to v7. A v5 decline suppresses only that earlier offer.
v7 is withdrawn	Hide v7 from new decisions. A replacement must be designated and compatible. Already-applied v7 remains the baseline. An earlier admission follows the policy below.
Auditor chooses Apply	Record an intent. Show "Application pending" until the owner confirms success or failure. Keep the last confirmed baseline while the outcome is unknown.

Who wins if v7 is withdrawn while it is on screen?

Proposed policy for Product. For each command or retry, the manager checks current decision rights on the file and matches the receipt to the firm, user and file. Bind request ID, payload, file revision and nonce in the firm DB. The authority checks target and epoch, orders admission with withdrawal per track and records the nonce outcome. Withdrawal first: reject and refresh. Admission first: the intent can commit even after withdrawal if its short-lived grant is valid and the file revision matches. Expiry requires a fresh view. Retries reuse the nonce. Return committed results and reject changed payloads. Atomically advance the file revision and store decision, grant, result and outbox. A grant alone cannot change a file.

Security controls across services

Use TLS in transit and regional KMS keys for evidence, queues and logs at rest. Verify encryption of firm databases and backups with their owner. Give each workload a least-privilege IAM role with temporary credentials. The worker calls the owner's API, which checks firm and file scope. The worker cannot read workpaper tables. Log actor, file, action and outcome regionally. Omit payloads, prompts and credentials.

Which part benefits from a model?

Grounding. Figure 2 uses the exact baseline and target. An addition in v5 reversed by v7 drops out of the v4-to-v7 diff. Rules handle simple diffs. In long releases, a model may group related edits across sections. Editors own wording and material exceptions. The model sees no tenant data and has no tools.

Release gate. Editors label whole held-out releases by market and language. Include reversals, wrong topics and polarity flips. Require all critical changes and zero unsupported claims. In a blind crossover trial, ship model grouping only if auditors find changes faster than with rules, with equal or better accuracy and within budget. A valid group can still mislead.

When would a different AI technique earn its cost?

Retrieval. If Product adds methodology explanations, resolve known references by ID. For unlinked guidance, test Bedrock Knowledge Bases with OpenSearch hybrid search over approved passages filtered by market, language and effective date. Require an applicable citation and archive its edition and passage. Otherwise withhold that explanation.

Model changes. Consider fine-tuning with QLoRA on compatible weights only for recurring, labeled grouping errors. Recursive FCoT and voting add calls without verifying facts. Use one bounded call. Revisit agent skills or MCP if several assistants need the same editorial procedure or tools.

From template diff to audit summary

Create one job per template comparison

On publish, warm common pairs. On a regional miss, enqueue once per product, track, base hash, target hash, locale and policy version. The pending badge is already visible while the first summary is prepared.

Ground each claim in the two approved versions

Fetch v4 and v7 by key. Verify both hashes. The existing JSON comparator emits net edits. For each, keep the source path, operation, scope, before/after values and permitted topics.

C17	**/inventory/signoffRequired**	**false to true**	Topics: inventory, review
C18	**/inventory/exceptionReason**	**absent to present**	Topics: inventory, exceptions

Rules summary by default

Typed edits select approved phrases. Use field labels and values from the verified pair. Hold a summary for editor review if a material edit lacks a safe mapping.

When each edit has one topic, group in code and skip inference.

Optional grouping in Amazon Bedrock

For long releases with several valid topics, send only typed facts, allowed topic IDs and a versioned glossary to Nova 2 Lite via Converse. Treat labels as data. Tools are disabled.

Output contract (illustrative group)

```
{"topic":"inventory","ids":["C17","C18"]}
```

What the auditor can read

“Inventory sign-off is now required.” **[C17]**
“Inventory now has a field for the reason for an exception.” **[C18]**

Presence alone says nothing about whether an entry is mandatory.

Validate before promoting a plan

Require valid JSON, every expected ID exactly once and a permitted topic for each ID. Code renders every sentence in the user’s locale. Reject a missing C18, an invented ID or extra prose. On timeout, excess input or budget exhaustion, keep the rules revision.

Preserve the summary the auditor actually saw

Store exact text, sentence links, diff, hashes, glossary, policy, prompt, model ID and checks in versioned S3 under Object Lock for the agreed retention period. Verify the regional copy before promotion. A late rules job cannot replace a promoted revision.

The regional API records the artifact version, file revision, catalog epoch, user, locale and time in the firm database before returning the text. A November review retrieves the March artifact and traces its sentences back to C17 and C18.

Evolution and recovery in production

Roll out without a maintenance window

Install tables, outboxes and create/open/decision hooks before the inventory watermark. Assume cheap ID listing. Capture metadata on opens and backfill resumably (~60 s/file). Exclude volatile values. Verify baselines and declines against owner history. Flag unknowns and reject stale repairs. Before rollout, verify that another firm's file or receipt yields no data or write and a read-only user cannot decide. On an isolation failure, disable affected API routes and investigate.

After launch: expand, migrate, contract

Expand. If pending-only pages exceed 500 ms at p95 for 5 min and query plans confirm sparse scans, build a pending-file index in the firm's region from verified metadata. Keep authorization on every read. For event schema changes, deploy consumers that read both versions before switching producers.

Migrate. In one small and one large firm per region, run the new query in shadow. Compare file IDs and offered versions at matching catalog and firm revisions, including declines and withdrawals. Enable it by firm flag, then widen the rollout only with matching results, p95 below 500 ms and projected monthly cost below X. Fall back to the live query if the index lags. Disable the flag on a mismatch or regression.

Contract. After the rollback window, retire obsolete event readers once all producers have switched and replay of retained events succeeds. Preserve archived summaries and view receipts.

Failure policy by dependency

Failure	Recovery behavior
Worker crash	Persist firm/file/release jobs and reuse the load ID per file revision. Owner deduplicates and conditionally commits repair and status. Renew leases. Acknowledge only after durable completion.
File API timeout	Trial deadlines: 3 s connect, 120 s server, 125 s total. Query the original ID. Hold the slot until completion or enforced cancellation. Reconcile ambiguous outcomes.
Transient errors	Three file attempts total across SDK and queue. Retry network errors, 429 and transient 5xx with exponential backoff, full jitter and Retry-After. Dead-letter permanent or exhausted failures. Replay with the same key.
Catalog or LLM outage	Open the circuit, then probe. Hold admissions and label stale reads (1 s total, one retry). Bedrock: 15 s/attempt, one budgeted retry on 429, 503 or timeout. Serve archived rules on failure.
Apply status unknown	Query the intent ID. Keep it pending until confirmed. Only success advances the baseline. Reconcile if status or idempotent dispatch is unavailable.

Failover and regional recovery

Use two zones per region. Route to healthy API replicas as ECS replaces failed tasks. Five peak replicas tolerate one loss. Throttle on zone loss if needed. Verify DB standby and promotion with its owner. On region loss, pause affected firms, restore in-region, replay events and reconcile receipts. Agree RTO/RPO and drill recovery. A standby region needs residency approval and funding.

Capacity controls and proposed SLOs

Commit job pages with cursors. Schedule firms fairly. Bound Java 25 virtual threads before calls under the owner's 20 shared slots. Scale on load per ready replica. Throttle hot firms and pause backfill on DB saturation.

Per region over 30 days, target 99% of catalog commits visible via API within 10 s and p95 under 500 ms for 50 files. Tabs add up to 5 s plus response and render. Track commit-to-display, first summary, oldest job and unverified share. Reconcile nightly. Drill zone loss, redelivery, lost replies, withdrawals and model outages.

Scaling limits and monthly cost

Which limit binds first?

Workload	Proposed operating point and what to validate
Releases and reads	40 × 52/12 ≈ 173 releases/month. Mean product: 20,000 files. At 50 files/page, the median firm fits on one page and the largest needs 800 for a full scan. List pages: 4k firms × 10/day × 20 days = 0.8M. Polls: 100 tabs × 8 h × 20 days × 720/h = 11.52M. Test sparse lists alongside small firms.
Interactive capacity	Assume a mixed 60 requests/s peak in one region. Target 15/s per replica only if a canary sustains 25/s at p95 ≤ 500 ms on the priced task size. Four serving replicas, five for one failure, six during rollout. Eight connections across all pools per replica means up to 48 list sessions in that region, plus write capacity. Load-test the mix and connection waits.
Slow file opens	20 owner slots × 60 s/min ÷ 60 s/load = 20 loads/minute. For 800,000 unknown files: 13,333 call-hours, about 27.8 days at full utilization or 55.6 days at 50%. Extra workers cannot raise the owner's limit.

Monthly scenario in USD / Frankfurt and Canada Central

On-demand list rates checked 10 Nov 2026. Assume Linux/x86 ECS Fargate, 730 h/month, two regional deployments and gross paid usage before tax, credits or discounts. Per-region volumes below are sizing assumptions.

Cost driver	Monthly units and assumptions	USD
ECS Fargate	2 API tasks/region (1 vCPU, 2 GB) $162.16. Frankfurt peak 120 task-h + 1 rollout task-h/region $6.92. One 0.5 vCPU, 1 GB worker/region $40.54.	209.63
ALB, IPv4, transfer	1 ALB and 1 average LCU/region $50.04. Four public IPv4 addresses $14.60. Per region: 25 GB internet out + 1 GB template transfer to the other region $4.54 combined.	69.18
S3, SQS, KMS	Per region: 25 GB S3 Standard, 5k PUT and 50k GET ($1.34 combined). 100k SQS units ($0.08). One KMS key and 50k calls ($2.30).	3.72
CloudWatch	Per region: 10 GB log ingestion, 5 GB-month archived, 10 custom metrics and 5 standard alarms.	19.13
Bedrock Nova 2 Lite	Global profile via US East, template facts only. 173.33 × 4 baselines × 2 locales × 1.25 attempts × (3k input at $0.30/M + 500 output at $2.50/M).	3.73
AWS usage subtotal	Sum at full precision. List traffic and polling run on the priced API and ALB.	305.38
Internal assumptions	Shared SQL, backups, platform and network allocation $150. 8k owner loads × $0.01 = $80. Operational review 2 h × $60 = $120.	350.00
Monthly envelope	**($305.3781 + $350) × 1.20 = $786.4537. Reserve $787/month for planning.**	**786.45**

Budget gate. Confirm X, owner cost/load and shared allocations with Finance. Full migration adds $8,000 before retries if $0.01/load holds. Each $0.01 increase per steady-state load adds $96/month including reserve. If each region needs a dedicated Multi-AZ db.t4g.medium PostgreSQL deployment with 20 GB gp3, add $222.26/month before reserve. Pace migration from headroom only while projected spend stays below X.

Rate evidence. See the companion cost workbook for units, regional rate cards and an editable X. AWS Price List Bedrock pricing

Worker implementation status

11 NOVEMBER 2026 / JAVA 25 / SPRING BOOT 4.1.2

This appendix tracks the background worker in Figure 1. The Java demo and full coverage gate are in main. Issue #9 documents the implementation and contributor workflow.

Implemented behavior and verification

Checkpoint	Contract and evidence
Shared admission Issue #1 / merged	Two workers share a JVM semaphore with a tested peak of three operations. A caller timeout leaves its permit reserved until a terminal outcome is confirmed or dispatch is ruled out.
Durable planning Issue #2 / merged	PostgreSQL commits each bounded page of jobs and its next cursor together. OperationId combines firm, engagement, publication and engagement revision. Tests cover resume, replay and rollback.
Downstream recovery Issue #3 / merged in PR #7	The client allows three submissions per execution with jittered backoff and Retry-After. It reconciles lost replies by OperationId and retries only after confirming the earlier attempt had no effect and can no longer execute.
Runnable demo Issue #4 / merged in PR #10	The demo resumes from a PostgreSQL checkpoint and completes seven jobs across four pages. The report measures peak concurrency, duplicate effects and reconciliation after a lost reply.
Code quality Issue #12 / merged in PR #13	Issue #8 introduced SonarQube in CI. Issue #12 requires 100% line and branch coverage in JaCoCo and 100% overall in SonarQube. A unit-only probe verifies that the gate rejects lower coverage.
Developer guides Issue #9 / review branch	The GitHub Wiki guides contributors through branch types, issue tracking, the project board and CI. Versioned diagrams explain the runnable Java worker and the proposed regional system.

Deployment boundary

The demo executes persisted planning jobs in one JVM. Production requires a job consumer, authoritative downstream status and shared admission. Figure 1 shows the proposed regional SQS flow. Coverage verification runs independently in CI.

> **Documentation and Wiki**
>
> Open the GitHub Wiki and keep it aligned with the versioned guides.

Versioned implementation references

Worker design | Java worker architecture | Regional system proposal | Code quality